

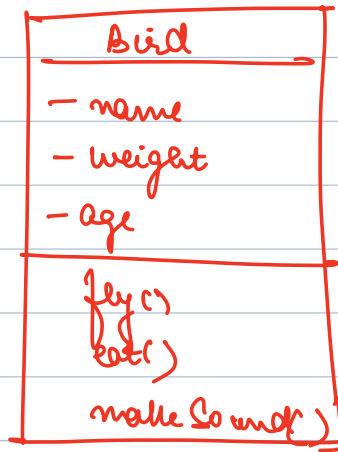
SOLID-2

Will start at
9:10 PM

Agenda:

- ① Liskov's Substitution Principle
- ② Interface Segregation Principle
- ③ Dependency Inversion Principle
- ④ Dependency Injection

→ V0

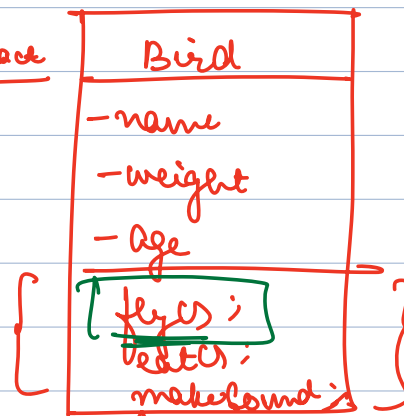


⇒ ~~not~~

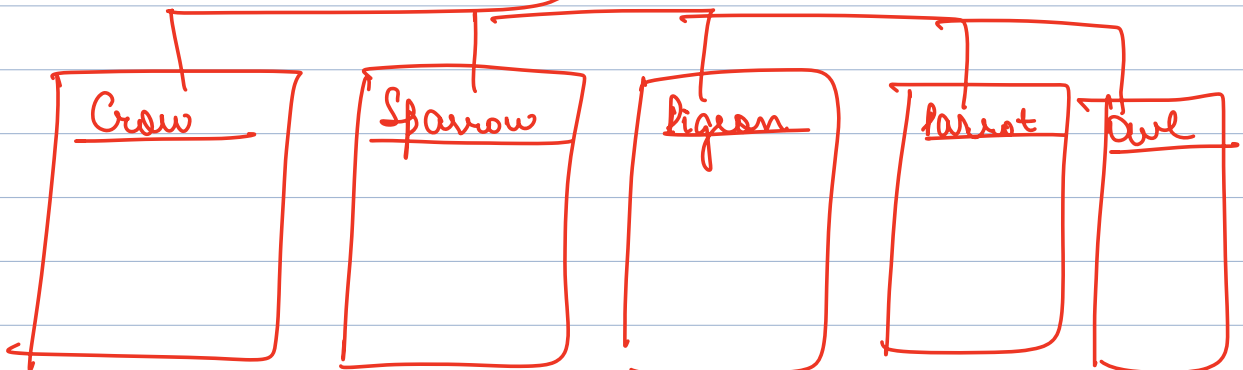
⇒ ~~not~~

→ V1

abstract



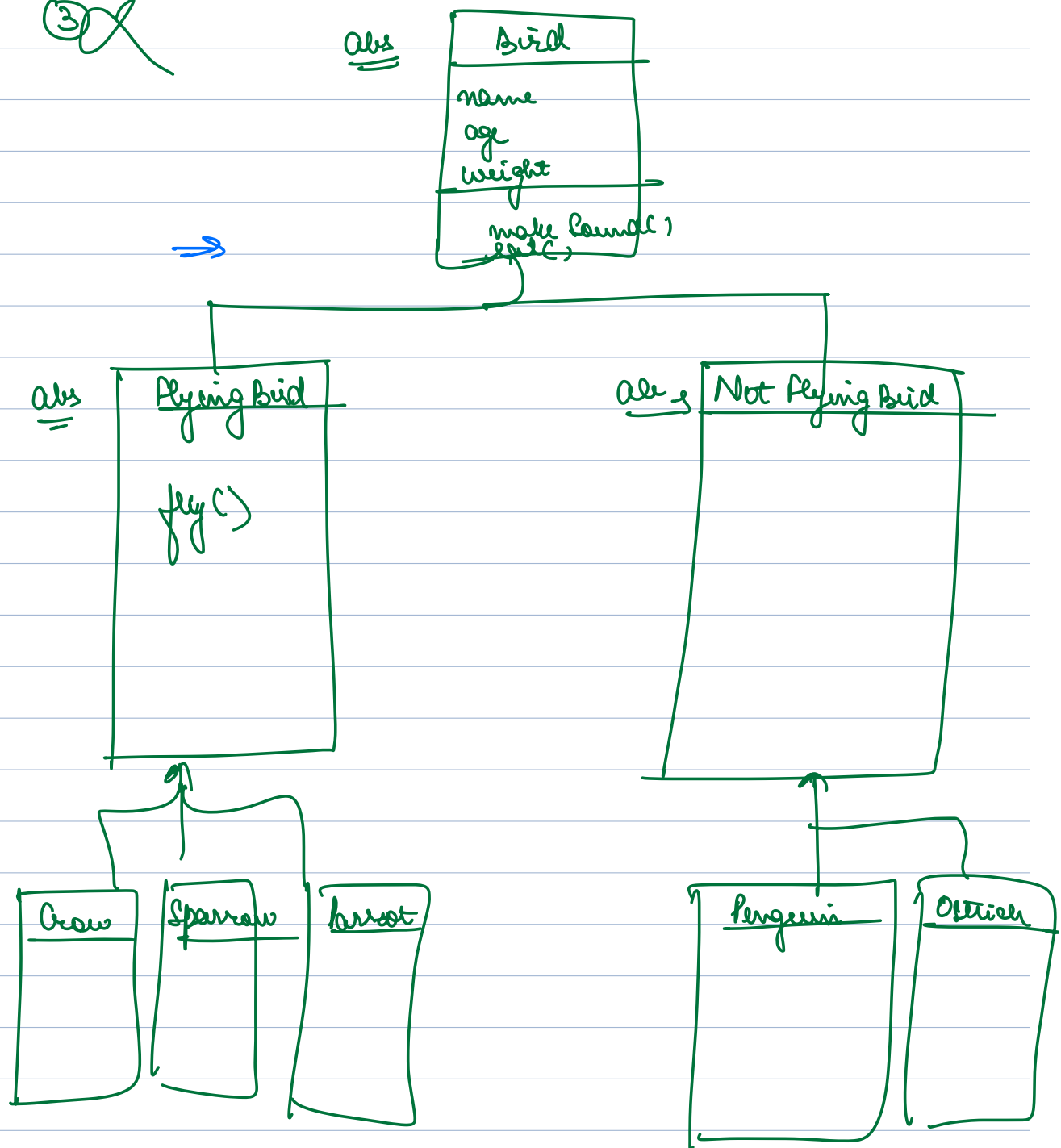
not all birds
can fly



How to handle not all birds can fly?

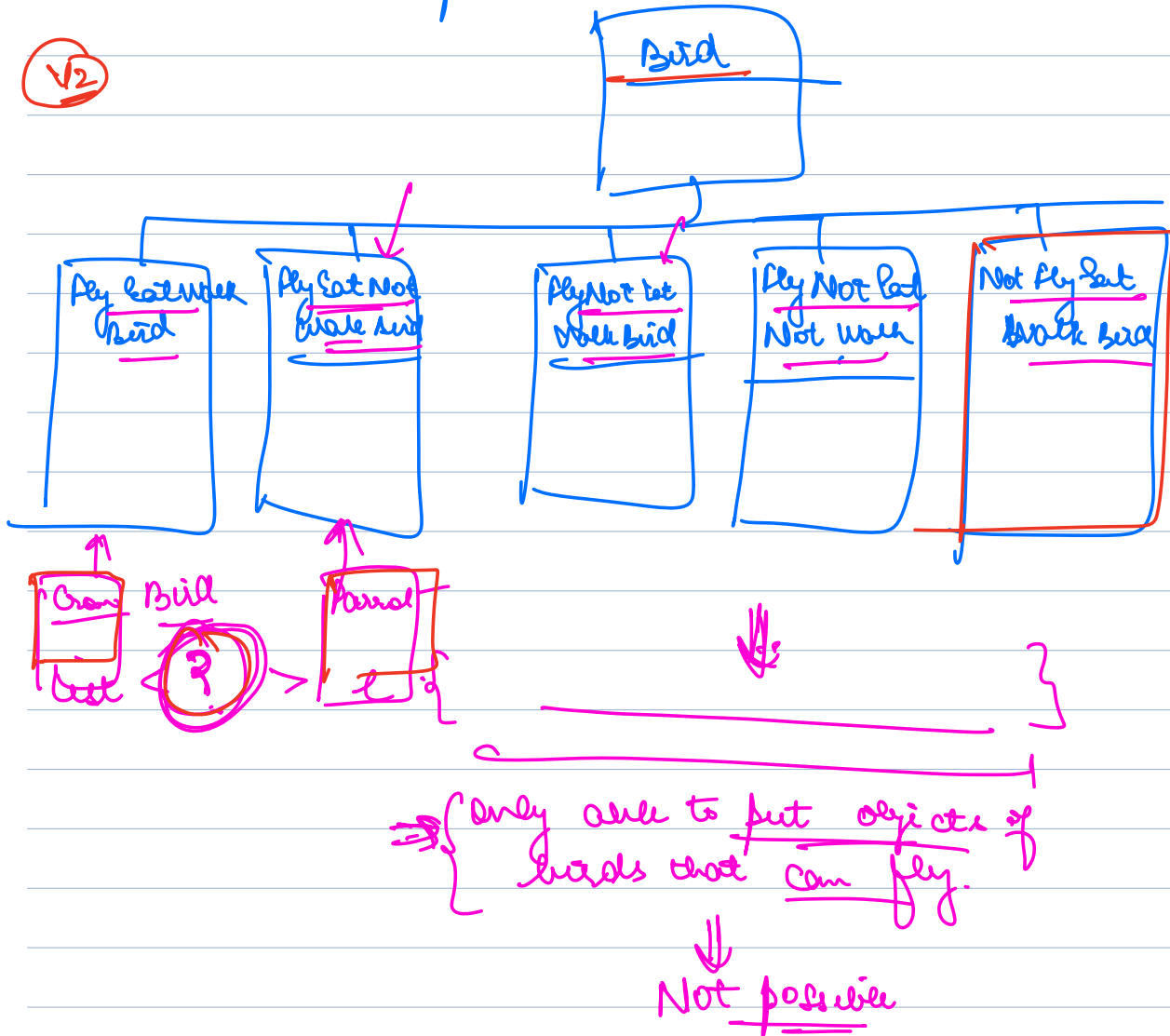
- ⇒ {
- ① In the fly method throw an exception
 - ② Keep the fly method empty.
- }

③ X



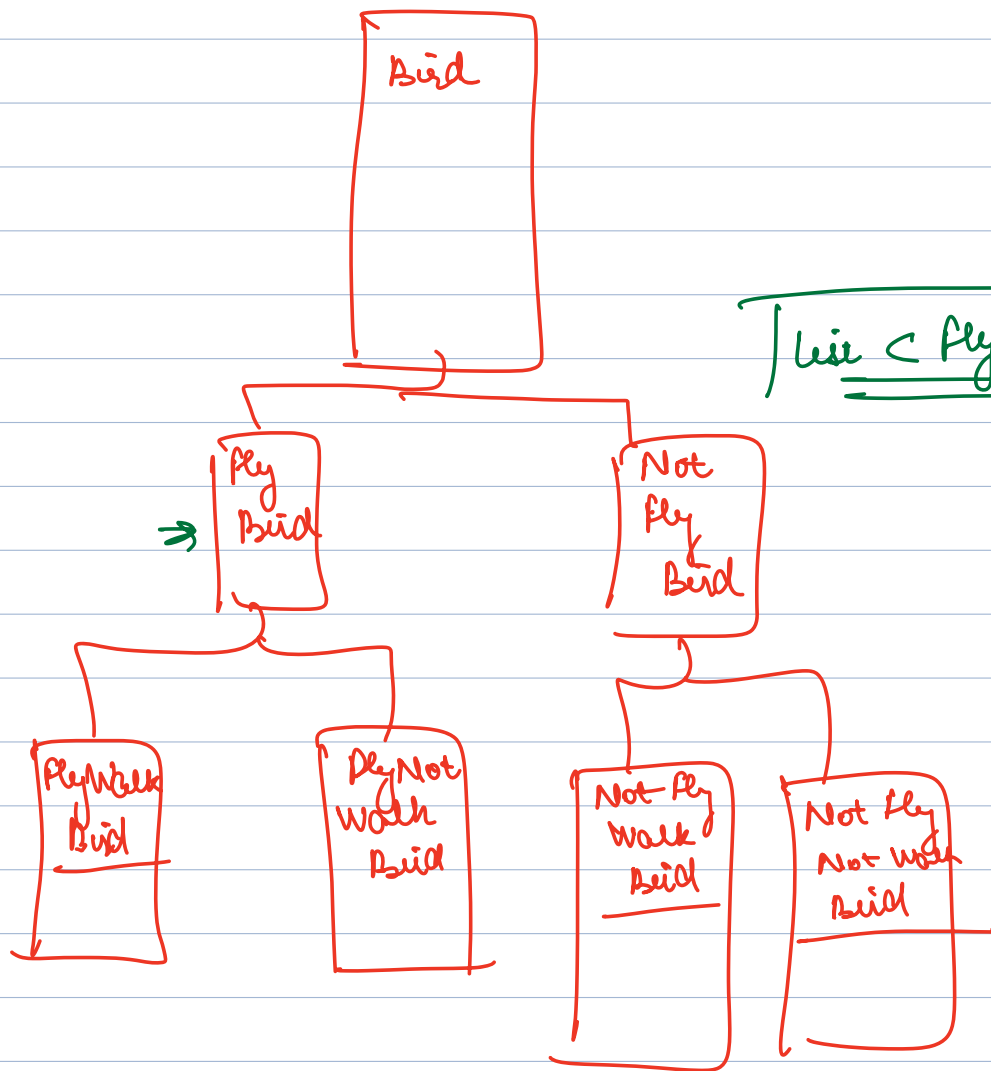
Problems with above

③ ⇒ Class Explosion

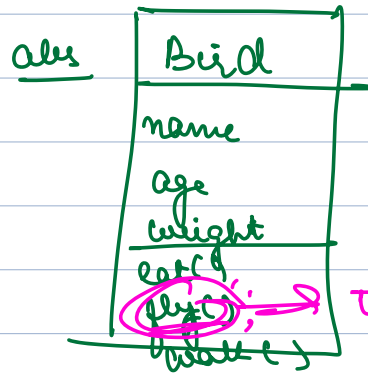


① Class Explosion

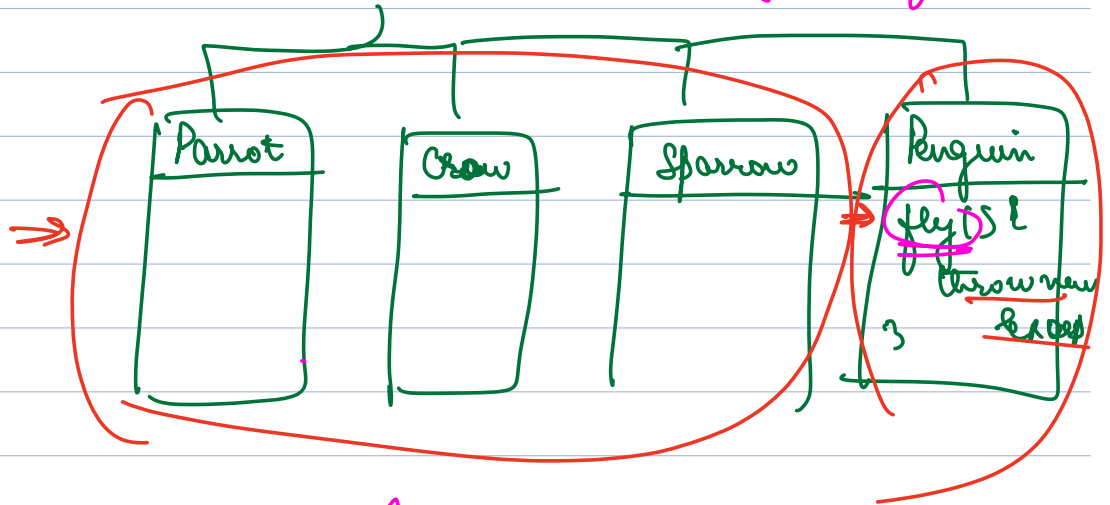
② Inability to recognize all instances that demonstrate the particular behaviour.



②, ①



this method will make corresponding bird fly.



Client {

private {

list <Bird> l;

penguin;

{ for (Bird v: l)
v.fly();

minimize
surprises for
your clients.

}

→ We would want that if an entity has a particular method, it must support that method.
→ Violates LSP

⇒ Violates Liskov's Substitution Principle

LSP

Objects of every child class should be AS-IS Substitutable with a variable of parent class. To reduce surprises

Bird b = new Penguin() ;

b.fly()

→ if (b.fly.didNothing)
 cout << "Bird can't fly".

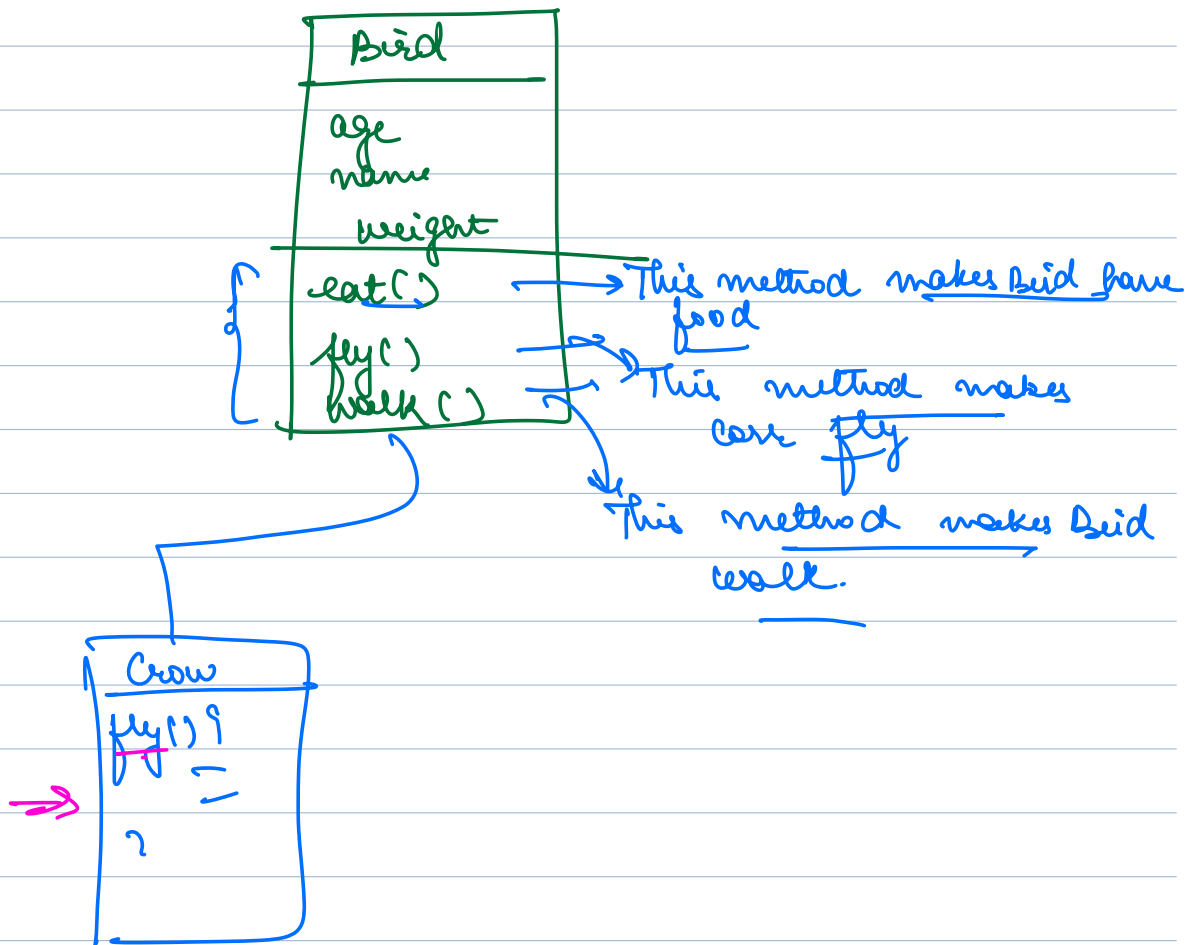
b Eat()

b.doSomething()

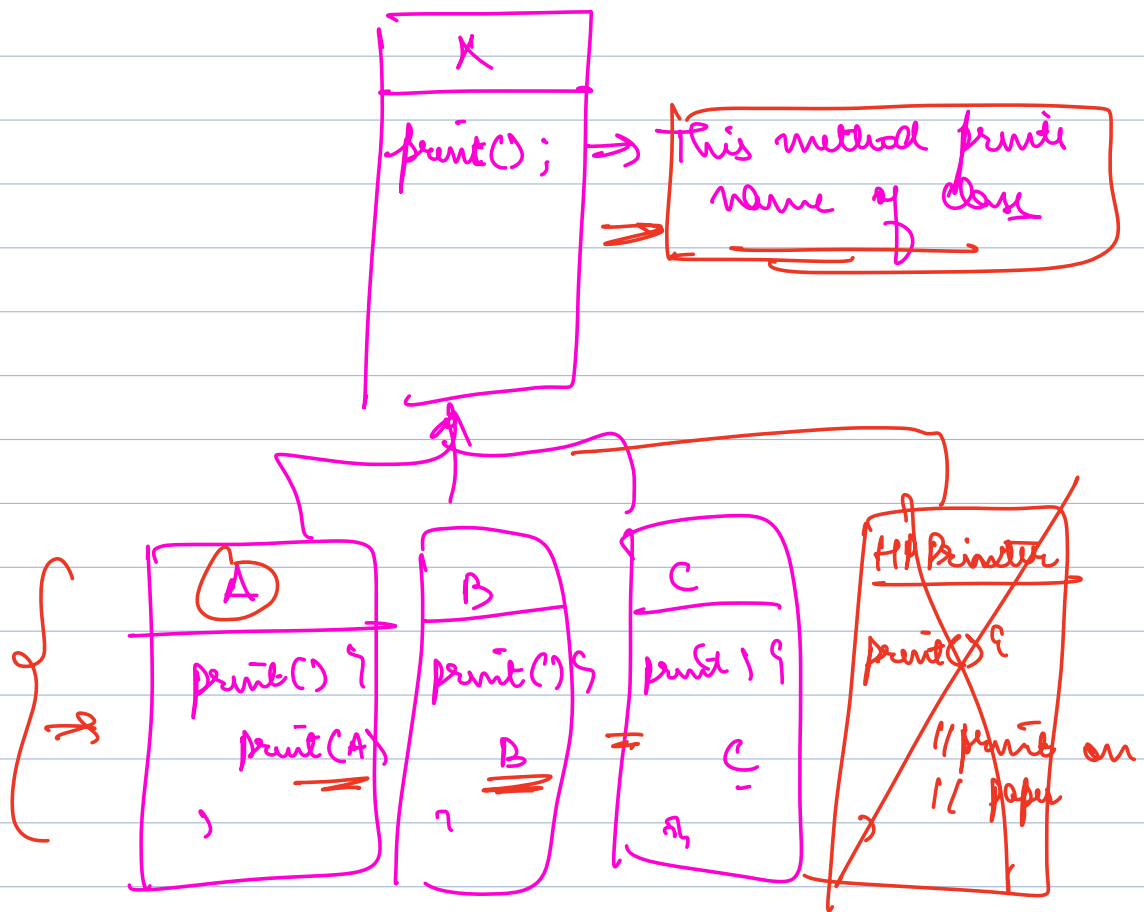
⇒ When I put an object of a child class, it should require me to make any changes to code ahead.

⇒ No child class should require any special treatment

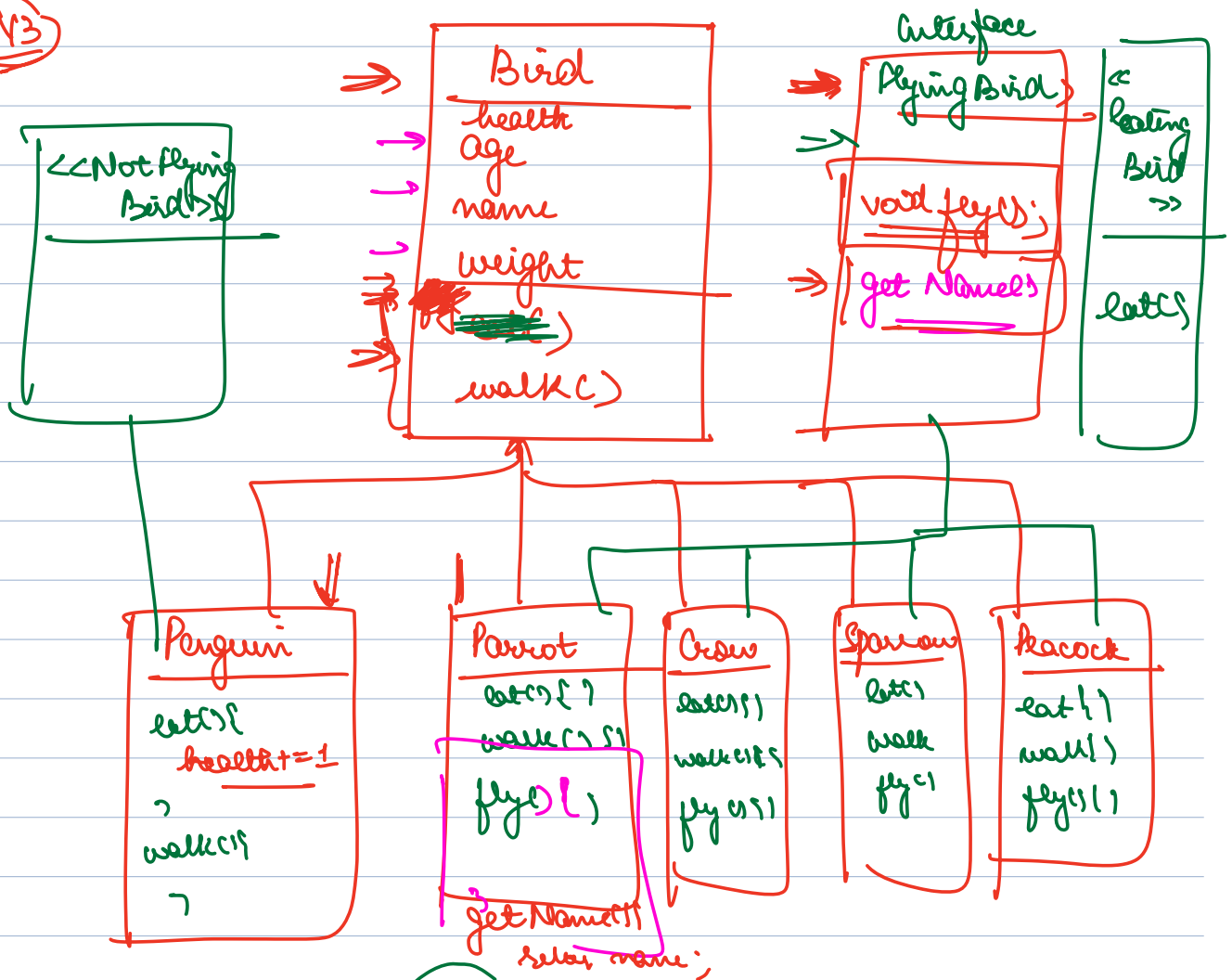
⇒ Objects of a child class should behave exactly how their parents expect them to.



→ Methods should be overridden to do the same thing differently, but not to do a diff thing.



V3

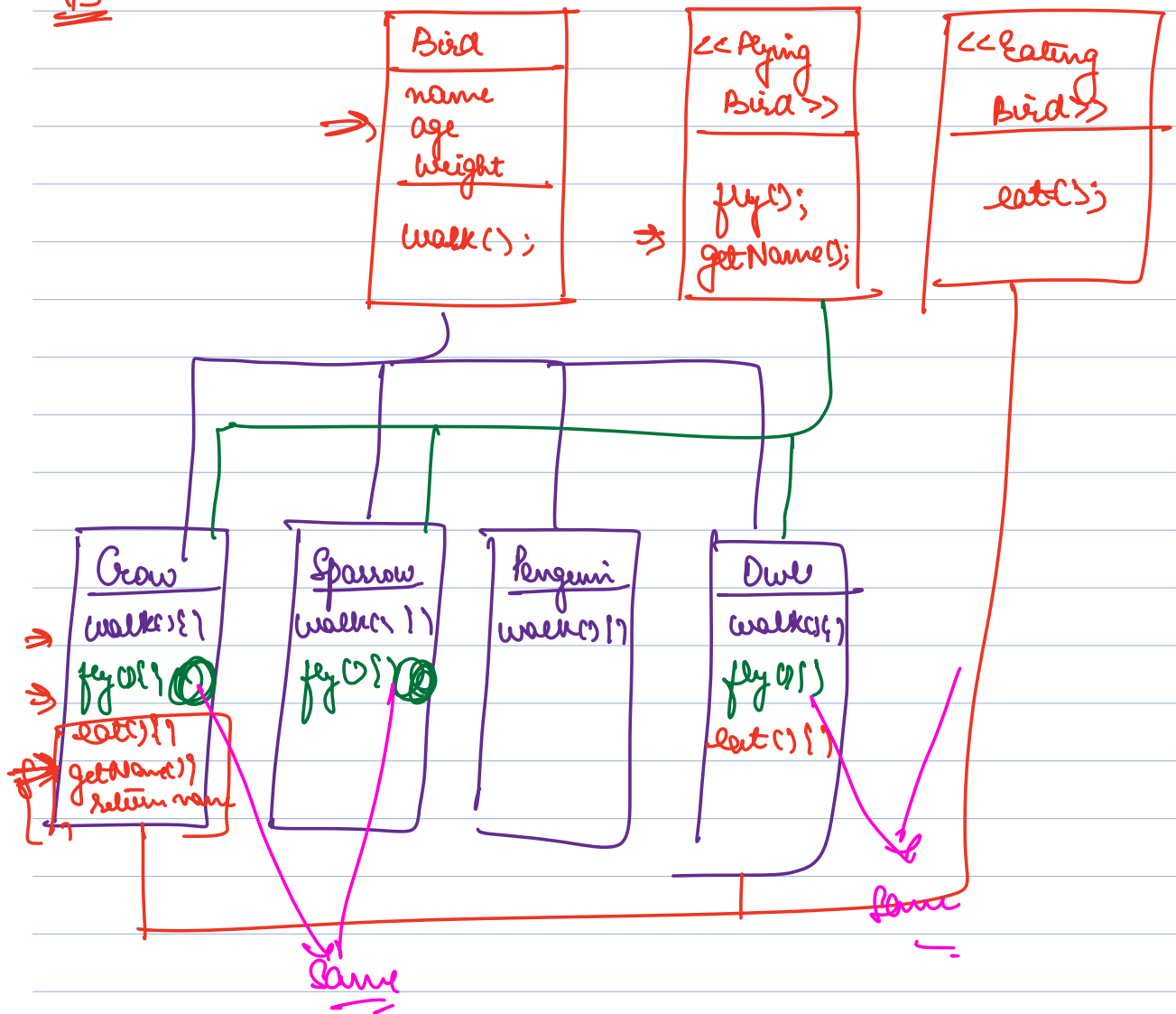


List <FlyingBird> = { ~~Penguin~~, Crow, Sparrow, Peacock },

$$\Rightarrow 10 \text{ V/S } 2^{10}$$

$$10 \text{ V/S } 1024$$

V3



Eating Bird `e = new Crow()`
`e.eat()`

`Crow c = new Crow()`
`list <Eating Bird>` $\Rightarrow$ `fly(*Flying Bird)`
 $\Rightarrow$ `fly(c)`

Bird

let <feature> ∈ {EAT, fly}

if bird. features. contains (fly)
flying Bird = bird.

Thank

⇒ So much common in industry that a special name is given to such interfaces
⇒ Functional Interfaces / Lambda function

(a, b) → return a+b;

do (a, b) → {
 //
};

Functional
Interfaces →

Runnable → run

Callable → call

Flying Bird →

Eating Bird →

→ SRP for interfaces

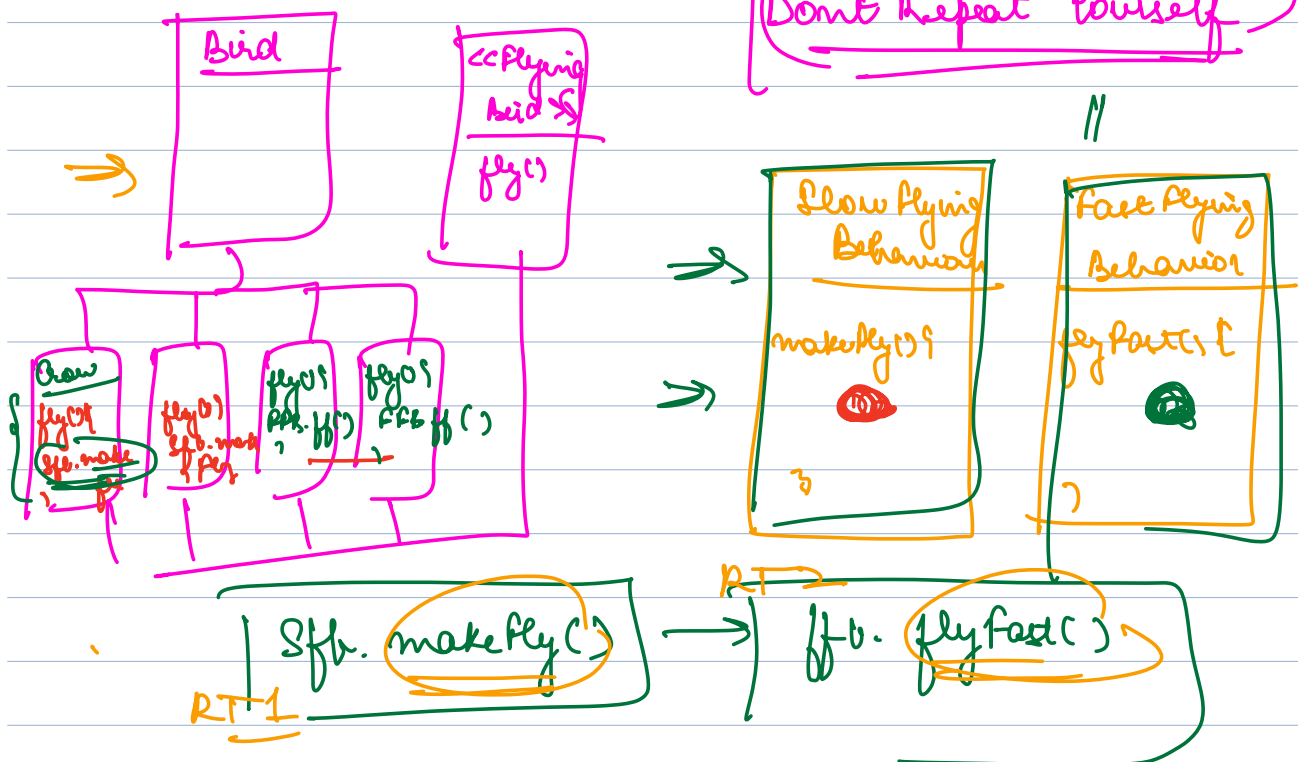
Does every bird fly in a diff way?

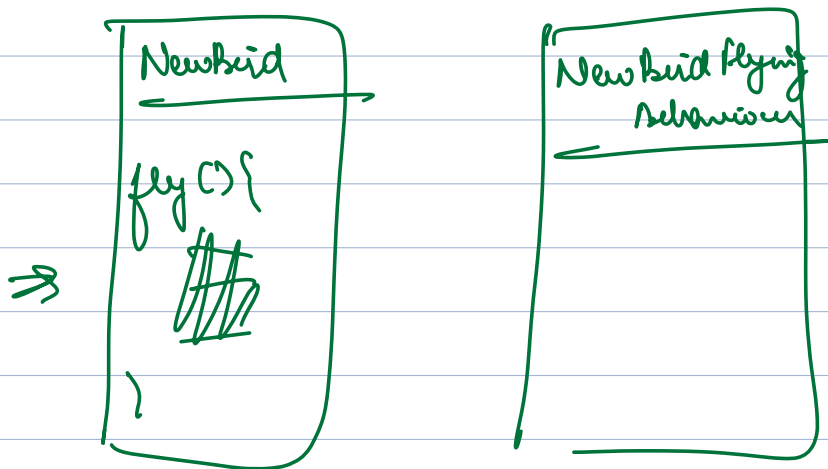
No

→ sparrow and crow fly in same way

→ parrot and peacock fly in same way.

(Don't Repeat Yourself)





Tradeoff

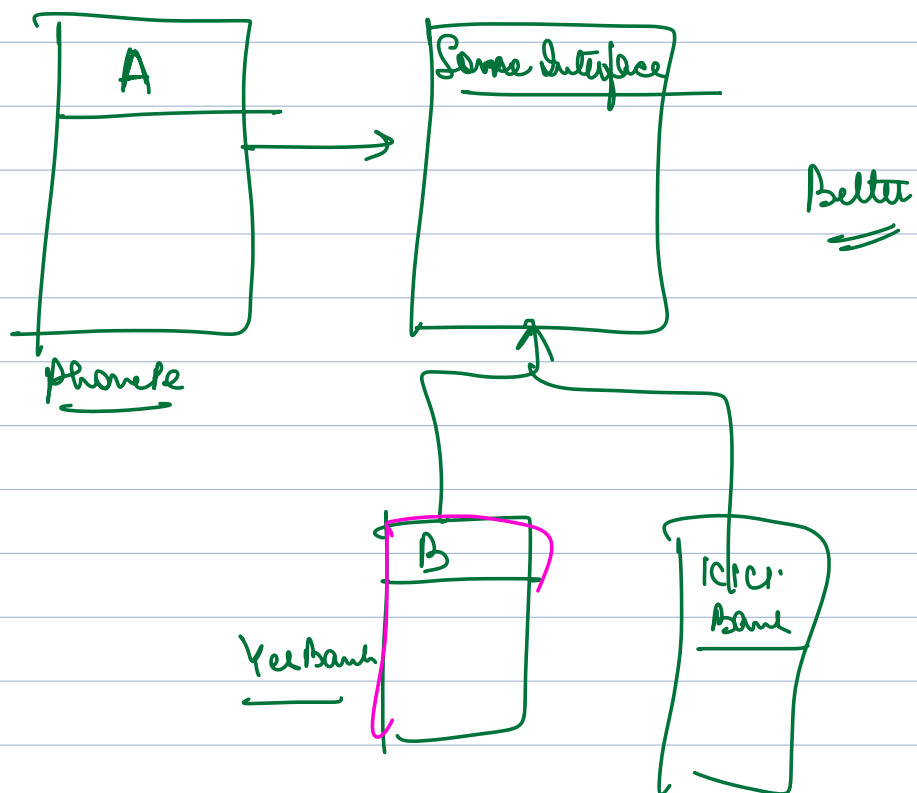
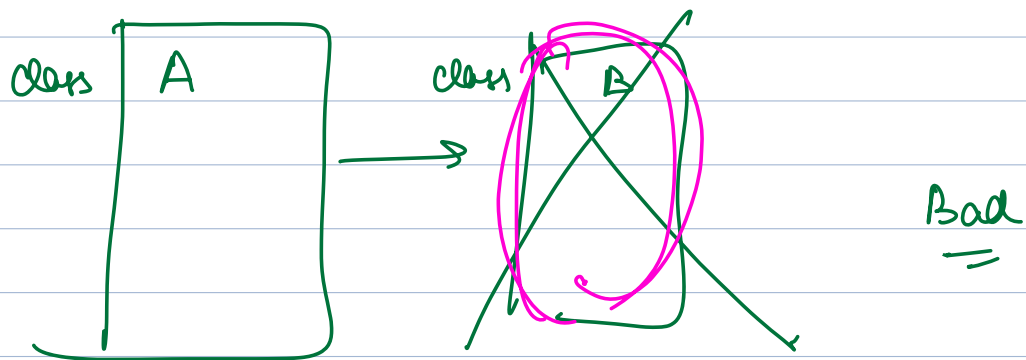
Pro: No code dupⁿ ⇨

Con: More Classes

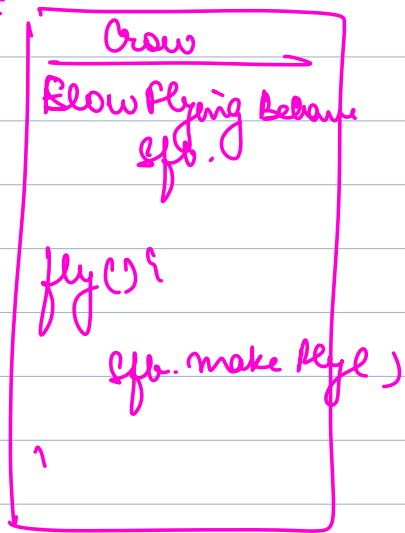
⇒ Code to an interface
not to an implementation

Dependency Inversion

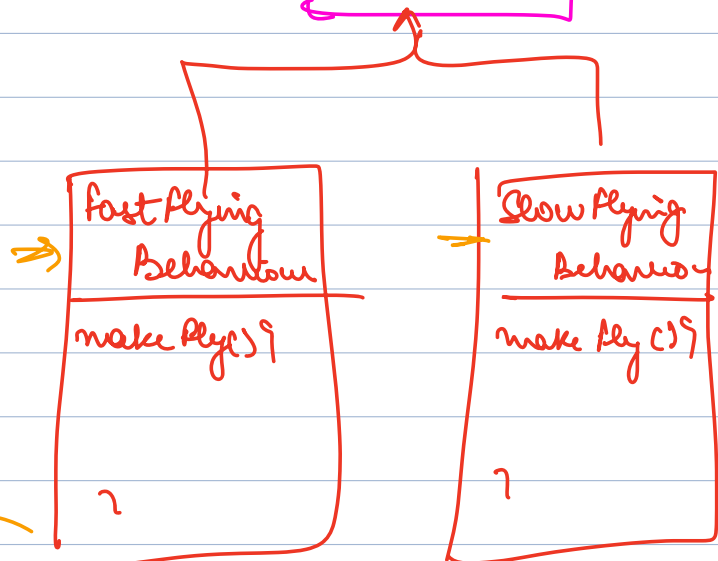
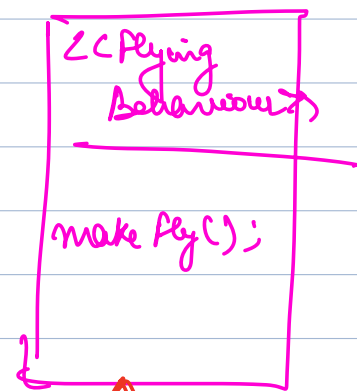
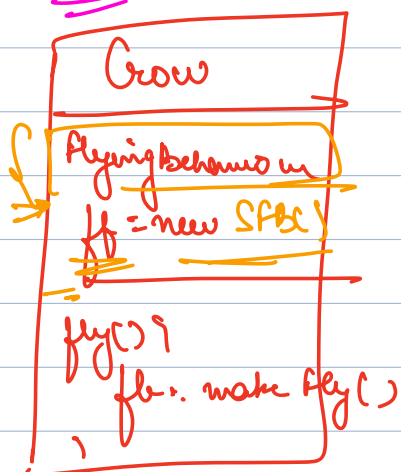
(non abstract)
: No 2 concrete classes
should ideally be dependent
on each other directly.
They should depend on
each other via an interface..



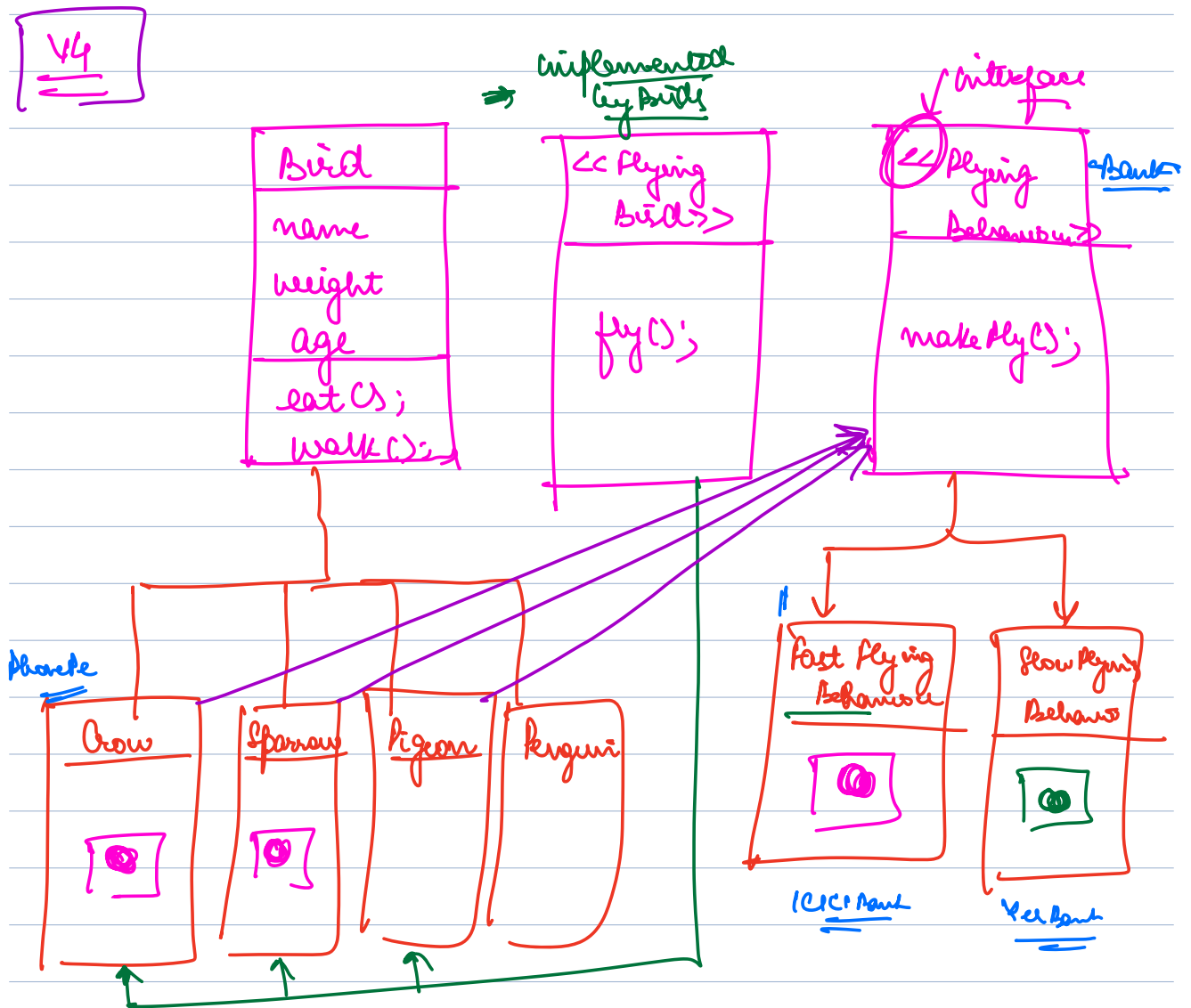
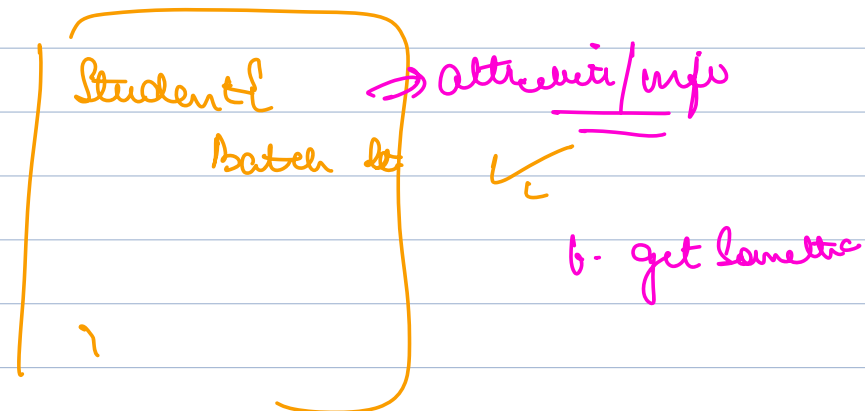
Earlier



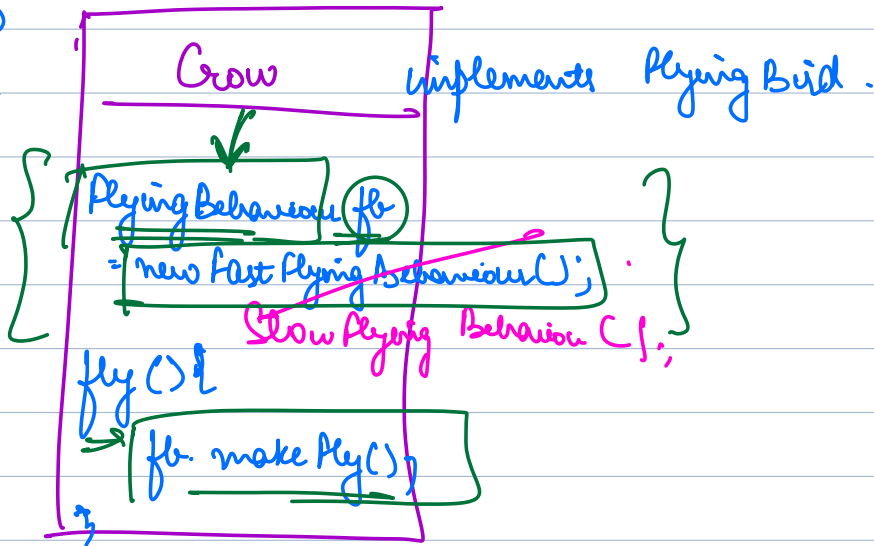
Now



Interfaces ⇒ Loose Coupling

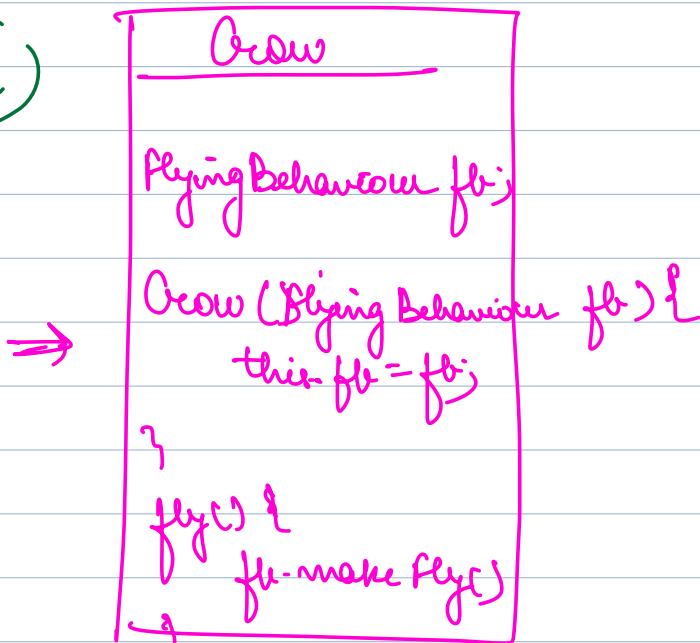


Strongly
Tied



- ⇒ Flying Behaviour is a dependency of Crow.
- ⇒ Today, dependency is being created within the class itself.

Q5



Dependency Injection
When an object of a dependency isn't created inside the class but passed to it via the client of that class.

Client {

Crow c = new Crow (new fast flying Behaviour())

?

H/W

Draw VO to VS yourself

30 mins